

机器学习与数据挖掘 作业三实验报告

姓名：庄仕豪 学号：23336355

一、算法描述

（一）K-Means 算法流程

K-Means 是一种基于距离的无监督聚类算法，其目标是将 n 个样本分为 k 个簇，使得簇内样本之间的距离尽可能小，簇间距离尽可能大。算法流程如下：

1. **初始化**：选择 k 个初始聚类中心 $\{\mu_1, \mu_2, \dots, \mu_k\}$ ；
2. **分配步骤（E 步）**：对于每个样本 x_i ，计算其到各个聚类中心的距离，将其分配到距离最近的聚类中心所属的簇：

$$c_i = \arg \min_j \|x_i - \mu_j\|^2, \quad j = 1, 2, \dots, k \quad (1)$$

其中 c_i 表示样本 x_i 所属的簇标签；

3. **更新步骤（M 步）**：根据当前簇的分配情况，重新计算每个簇的聚类中心：

$$\mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i \quad (2)$$

其中 C_j 表示第 j 个簇中的所有样本， $|C_j|$ 表示簇中样本的数量；

4. **收敛判断**：重复步骤 2 和步骤 3，直到聚类中心不再发生变化或变化小于预设阈值，或达到最大迭代次数。

K-Means 算法的目标函数（损失函数）为：

$$J = \sum_{j=1}^k \sum_{x_i \in C_j} \|x_i - \mu_j\|^2 \quad (3)$$

该算法通过迭代优化使目标函数值不断减小，最终收敛到局部最优解。

（二）GMM 算法流程

高斯混合模型（Gaussian Mixture Model, GMM）是一种基于概率的聚类方法，假设数据由 k 个高斯分布混合而成。每个数据点由某个高斯分量生成，GMM 通过 EM 算法估计模型参数。

GMM 的概率密度函数定义为：

$$p(x) = \sum_{j=1}^k \pi_j \mathcal{N}(x|\mu_j, \Sigma_j) \quad (4)$$

其中， π_j 是第 j 个高斯分量的混合系数（满足 $\sum_{j=1}^k \pi_j = 1$ ）， μ_j 和 Σ_j 分别是第 j 个高斯分量的均值向量和协方差矩阵， $\mathcal{N}(x|\mu_j, \Sigma_j)$ 表示多元高斯分布：

$$\mathcal{N}(x|\mu_j, \Sigma_j) = \frac{1}{(2\pi)^{d/2} |\Sigma_j|^{1/2}} \exp \left(-\frac{1}{2} (x - \mu_j)^T \Sigma_j^{-1} (x - \mu_j) \right) \quad (5)$$

其中 d 是数据的维度。

使用 EM 算法训练 GMM 的流程如下：

1. **初始化**：初始化模型参数 $\{\pi_j, \mu_j, \Sigma_j\}_{j=1}^k$ ；
2. **E 步 (Expectation)**：计算每个样本 x_i 属于第 j 个高斯分量的后验概率（责任度）：

$$\gamma_{ij} = p(z_i = j|x_i) = \frac{\pi_j \mathcal{N}(x_i|\mu_j, \Sigma_j)}{\sum_{l=1}^k \pi_l \mathcal{N}(x_i|\mu_l, \Sigma_l)} \quad (6)$$

其中 z_i 是样本 x_i 的隐变量，表示其所属的高斯分量；

3. **M 步 (Maximization)**：根据后验概率更新模型参数：

$$N_j = \sum_{i=1}^n \gamma_{ij} \quad (7)$$

$$\pi_j = \frac{N_j}{n} \quad (8)$$

$$\mu_j = \frac{1}{N_j} \sum_{i=1}^n \gamma_{ij} x_i \quad (9)$$

$$\Sigma_j = \frac{1}{N_j} \sum_{i=1}^n \gamma_{ij} (x_i - \mu_j)(x_i - \mu_j)^T \quad (10)$$

4. **收敛判断**：重复步骤 2 和步骤 3，直到对数似然函数的增量小于预设阈值或达到最大迭代次数。对数似然函数为：

$$\log p(X|\theta) = \sum_{i=1}^n \log \left(\sum_{j=1}^k \pi_j \mathcal{N}(x_i|\mu_j, \Sigma_j) \right) \quad (11)$$

GMM 相比 K-Means 的优势在于：它是一个软聚类方法，能够给出每个样本属于各个簇的概率；同时能够捕捉数据的协方差结构，适应不同形状和大小的簇。

二、实验设计

（一）初始化方法

1、K-Means 初始化方法

本实验探索了两种不同的初始化方法：

1. **随机初始化**: 从数据集中随机选择 k 个样本作为初始聚类中心。该方法简单直接, 但可能导致算法收敛到较差的局部最优解。
2. **K-Means++ 初始化**: 这是一种改进的初始化方法, 能够使初始聚类中心尽可能分散:
 - 随机选择第一个聚类中心;
 - 对于每个样本 x , 计算其到已选择的最近聚类中心的距离 $D(x)$;
 - 以概率 $\frac{D(x)^2}{\sum_x D(x)^2}$ 选择下一个聚类中心;
 - 重复上述过程, 直到选出 k 个聚类中心。

K-Means++ 初始化能够显著提高聚类质量, 减少陷入较差局部最优的概率。

2、GMM 初始化方法

对于 GMM 模型, 本实验采用以下初始化策略:

1. 使用 K-Means 算法对数据进行预聚类, 将 K-Means 的聚类中心作为 GMM 的初始均值 μ_j ;
2. 混合系数 π_j 初始化为均匀分布, 即 $\pi_j = 1/k$;
3. 协方差矩阵 Σ_j 根据不同的实验设置进行初始化 (详见下节)。

(二) 协方差矩阵结构

在 GMM 实验中, 探索了三种不同结构的协方差矩阵:

1. **球形协方差 (Spherical)**: 协方差矩阵为对角矩阵且所有对角元素相等, 即 $\Sigma_j = \sigma_j^2 I$, 其中 I 是单位矩阵。这种结构假设各维度独立且方差相同, 计算最简单。
2. **对角协方差 (Diagonal)**: 协方差矩阵为对角矩阵但对角元素可以不同, 即 $\Sigma_j = \text{diag}(\sigma_{j1}^2, \sigma_{j2}^2, \dots, \sigma_{jd}^2)$ 。这种结构假设各维度独立但允许不同的方差。
3. **完全协方差 (Full)**: 使用完整的协方差矩阵 Σ_j , 不做任何限制。这种结构能够捕捉维度之间的相关性, 但参数数量最多, 计算复杂度最高。

(三) 实现细节

1、数据预处理

1. 读取 MNIST 数据集, 每个样本为 $28 \times 28 = 784$ 维的向量;
2. 对像素值进行归一化处理, 将其缩放到 $[0, 1]$ 区间;
3. 可选: 进行 PCA 降维以提高计算效率 (K-Means 没有使用, 但是 GMM 使用了)。

2、聚类精度计算

由于聚类算法得到的簇标签与真实标签可能存在排列不一致的问题，需要先找到最优的标签匹配。本实验使用匈牙利算法（Hungarian Algorithm）求解最优匹配，然后计算聚类精度：

$$ACC = \frac{1}{n} \sum_{i=1}^n \mathbb{I}\{y_i = \text{map}(c_i)\} \quad (12)$$

其中 y_i 是样本的真实标签， c_i 是聚类得到的簇标签， $\text{map}(\cdot)$ 是通过匈牙利算法得到的最优映射函数。

三、实验结果与分析

（一）K-Means 聚类结果

在 K-Means 实验中，我们对比了两种不同的初始化方法：随机初始化（Random）和 K-Means++ 初始化。实验在完整的 MNIST 训练集（60000 个样本）上进行，结果如表 1 所示。

表 1: K-Means 不同初始化方法的性能对比

初始化方法	训练 ACC	测试 ACC	收敛迭代数	最终损失
Random	0.5826	0.5937	76	2361084.75
K-Means++	0.5555	0.5598	46	2359941.29

对本次实验的观察：

1. **随机初始化的性能有时会更好：**随机初始化在训练集和测试集上都取得了更高的聚类精度（分别为 58.26% 和 59.37%），比 K-Means++ 初始化高出约 3-4 个百分点，这一结果可能是由于随机初始化偶然选择了更有利的初始质心位置。
2. **K-Means++ 收敛更快：**虽然 ACC 比随机初始化低一些，但 K-Means++ 初始化仅需 46 次迭代即可收敛，而随机初始化需要 76 次迭代。
3. **损失函数与 ACC 的不一致：**K-Means++ 初始化得到的最终损失值（2359941.29）略低于随机初始化（2361084.75），但其聚类精度反而更低。这说明损失函数（簇内平方距离和）的最小化并不一定对应于更好的标签一致性。

训练过程分析（见图 1）：从训练曲线可以看出，两种初始化方法的损失都在前期快速下降，随后趋于平稳。随机初始化的 ACC 在训练过程中持续上升并在后期稳定在较高水平，而 K-Means++ 初始化虽然收敛更快，但 ACC 的最终值较低。

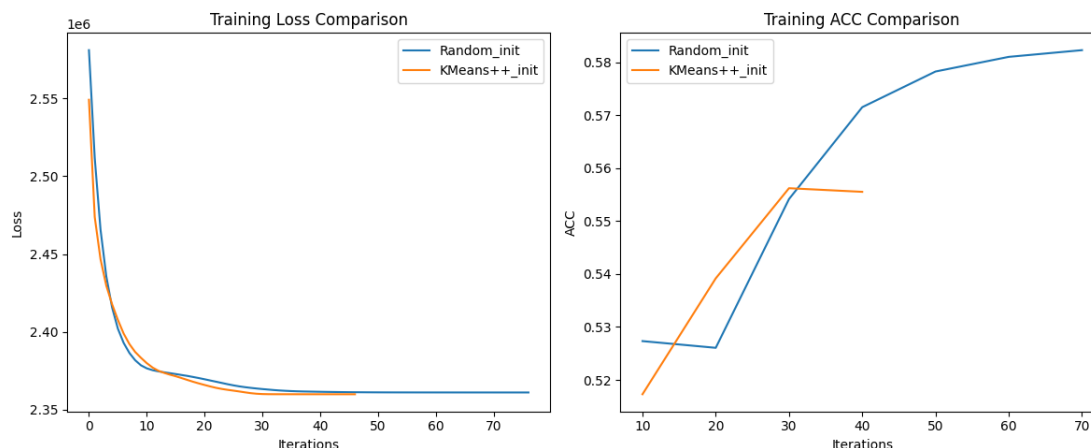


图 1: K-Means 训练过程对比（损失曲线与 ACC 曲线）

（二）GMM 聚类结果

1、不同协方差矩阵结构的影响（固定 K-Means++ 初始化）

为了研究协方差矩阵结构对 GMM 性能的影响，我们在使用 PCA 降维到 50 维后，对比了三种协方差结构：球形（Spherical）、对角（Diagonal）和完全（Full）。所有实验均使用 K-Means++ 初始化，结果如表 2 所示。

表 2: GMM 不同协方差结构的性能对比（K-Means++ 初始化）

协方差结构	训练 ACC	测试 ACC	训练时间 (s)	迭代次数	对数似然
Spherical	0.5402	0.5376	40.24	80	-592049.07
Diagonal	0.4217	0.4095	52.44	80	-518806.90
Full	0.5666	0.5763	126.15	150	-308797.24

本次实验发现：

1. **Full** 协方差性能在本次实验中最优：完全协方差矩阵在测试集上取得了最高的聚类精度 (57.63%)，同时对数似然值也最高 (-308797.24)，说明其能够更好地拟合数据的真实分布。
2. **Diagonal** 协方差性能在本次实验中最差：对角协方差的 ACC 仅为 40.95%，显著低于其他两种结构。这可能是因为对角协方差假设各维度独立，但 PCA 降维后的主成分之间仍存在一定的相关性。
3. **计算复杂度与性能的权衡**：Full 协方差虽然性能最好，但训练时间 (126.15s) 是 Spherical (40.24s) 的三倍多，同时收敛速度慢于其他两种结构。在资源受限的情况下，Spherical 协方差提供了较好的性能-效率平衡。
4. **对数似然与 ACC 的一致性**：在协方差结构对比中，对数似然越高的模型，其 ACC 也越高，两者呈现较好的正相关关系。

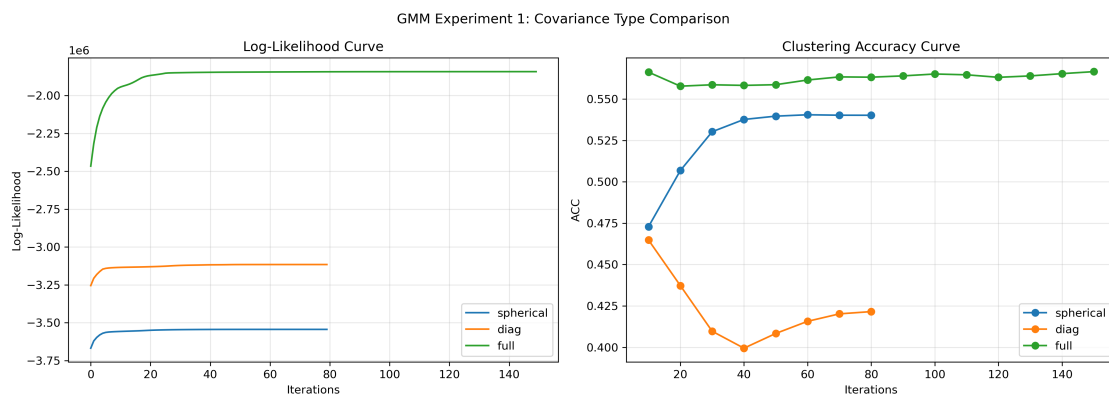


图 2: 不同协方差结构的 GMM 训练曲线

2、不同初始化方法的影响（固定 Full 协方差）

为了研究初始化方法对 GMM 性能的影响，我们固定使用 Full 协方差矩阵，对比了随机初始化和 K-Means++ 初始化的效果，结果如表 3 所示。

表 3: GMM 不同初始化方法的性能对比（Full 协方差）

初始化方法	训练 ACC	测试 ACC	训练时间 (s)	迭代次数	对数似然
Random	0.6337	0.6350	136.29	150	-314934.24
K-Means++	0.5666	0.5763	123.78	150	-308797.24

本次实验结果：

1. **随机初始化 ACC 显著更高：**随机初始化在测试集上的 ACC 达到 63.50%，比 K-Means++ 初始化（57.63%）高出近 6 个百分点。可能是因为随机初始化偶然选择了更有利的初始参数，而且此处的 Kmeans++ 初始化可能不是和 sklearn 中的完全一致，加上了自己的一些改动，导致效果不佳。
2. **对数似然与 ACC 的矛盾：**K-Means++ 初始化得到的对数似然值（-308797.24）高于随机初始化（-314934.24），说明其模型更好地拟合了数据的概率分布。然而，这并未转化为更高的聚类精度。
3. **收敛行为的差异：**两种初始化方法都在达到最大迭代次数（150 次）时停止，未完全收敛。从训练曲线（图 3）可以看出，两者的对数似然都在持续上升，且 K-Means++ 的对数似然始终高于随机初始化，说明 K-means++ 收敛更快，但 ACC 的变化趋势不同。

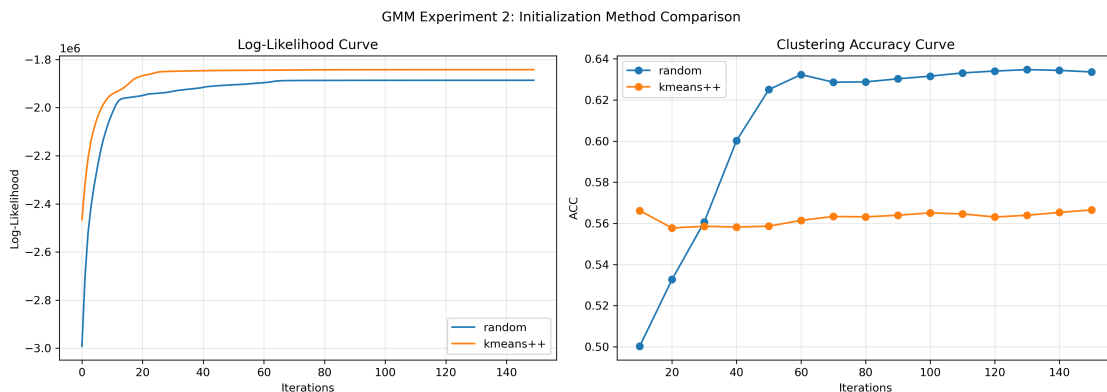


图 3: 不同初始化方法的 GMM 训练曲线

现象解释与分析:

这一结果可以从以下几个角度解释:

1. **优化目标的不一致:** EM 算法优化的是数据的对数似然, 即最大化 $\log p(X|\theta)$, 而不是直接优化聚类精度 ACC。对数似然衡量的是模型对数据概率分布的拟合程度, 而 ACC 衡量的是簇标签与真实标签的一致性。这两个目标并不总是一致的。
2. **局部最优与标签对齐:** GMM 是非凸优化问题, 不同的初始化会导致 EM 算法收敛到不同的局部最优解。K-Means++ 初始化虽然能找到更高的对数似然, 但其对应的簇划分可能与真实标签的对应关系较差。相反, 随机初始化可能偶然落入一个对数似然稍低、但簇与标签对齐更好的局部极值。
3. **簇的形状与分布:** K-Means++ 倾向于选择彼此距离较远的初始质心, 这可能导致某些类别被分散到多个簇中, 或多个类别被合并到一个簇中。随机初始化虽然没有这种“距离最大化”的偏向, 但可能更容易得到与真实类别分布相匹配的划分。
4. **匈牙利匹配的影响:** 聚类精度的计算依赖于匈牙利算法进行最优标签匹配。不同的簇划分方式会影响匹配的效果。K-Means++ 初始化可能产生大小不均的簇, 导致匹配时出现不利情况。

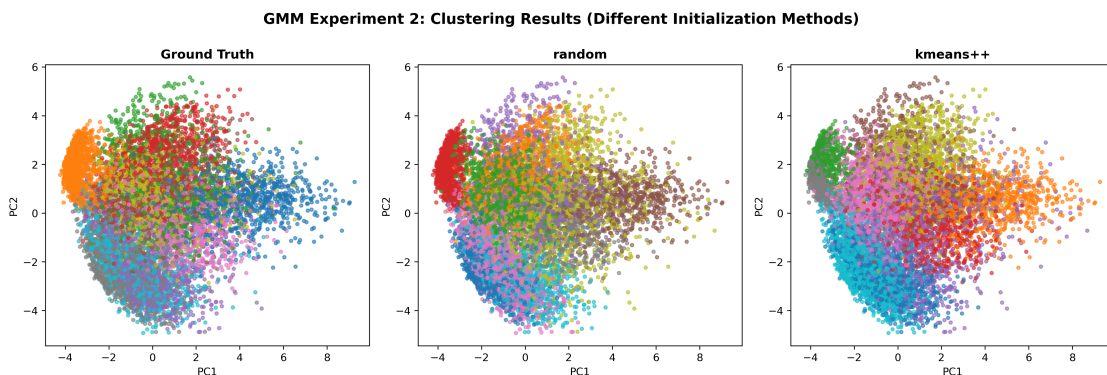


图 4: 不同初始化方法的聚类结果可视化 (PCA 2D 投影)

从聚类结果的可视化 (图 4) 可以看出, 随机初始化和 K-Means++ 初始化产生了不同的簇边界。虽然两者在视觉上都能较好地分离不同的数据点集, 但与真实标签的对应关系存在差异。

（三）算法比较与讨论

1、K-Means 与 GMM 的性能对比

表 4: K-Means 与 GMM 最佳配置的性能对比

方法	配置	训练 ACC	测试 ACC
K-Means	Random 初始化	0.5826	0.5937
GMM	Full 协方差 + Random 初始化	0.6337	0.6350

从实验结果可以看出，GMM（Full 协方差 + Random 初始化）在聚类精度上显著优于 K-Means，测试集 ACC 提高了约 4 个百分点（59.37% vs 63.50%）。

2、K-Means 与 GMM 的优劣分析

K-Means 的优势：

1. **计算效率高：**K-Means 算法简单，每次迭代只需计算距离和更新质心，计算复杂度为 $O(nkd)$ ，其中 n 是样本数， k 是簇数， d 是维度。在相同迭代次数下，K-Means 的运行时间远低于 GMM。
2. **易于理解和实现：**算法逻辑直观，目标函数明确（最小化簇内平方距离和），易于调试和优化。
3. **内存占用小：**只需存储 k 个质心（ $k \times d$ 的矩阵），而 GMM 需要存储均值、协方差矩阵和混合系数。
4. **适用于球形簇：**当数据的簇形状接近球形且大小相近时，K-Means 能取得较好的效果。

K-Means 的劣势：

1. **硬分配导致信息损失：**K-Means 采用硬分配策略，每个样本只能属于一个簇，无法表达样本属于多个簇的不确定性。
2. **假设簇为球形且等方差：**K-Means 隐含假设所有簇具有相同的形状（球形）和大小，这在实际数据中往往不成立。
3. **对异常点敏感：**使用平方距离作为度量，异常点会显著影响质心的位置。
4. **难以处理不同密度的簇：**当簇的密度差异较大时，K-Means 倾向于将大簇分割成多个小簇。

GMM 的优势：

1. **软聚类提供概率信息：**GMM 给出每个样本属于各个簇的概率（后验概率），提供了更丰富的信息，可用于不确定性量化。
2. **灵活的协方差结构：**通过选择不同的协方差矩阵（球形、对角、完全），GMM 能够适应不同形状、大小和方向的簇。实验表明，Full 协方差能够显著提高聚类精度。
3. **理论基础扎实：**GMM 是基于概率模型的，有明确的似然函数和 EM 算法的收敛性保证（收敛到局部最优）。
4. **可生成模型：**GMM 不仅可以进行聚类，还可以用于生成新的样本，或计算样本的概率密度。

GMM 的劣势：

1. **计算复杂度高：**E 步需要计算每个样本在每个高斯分量下的概率，M 步需要更新协方差矩阵。对于 Full 协方差，计算复杂度为 $O(nkd^2)$ 。实验中，GMM 的训练时间是 K-Means 的 2-3 倍。
2. **内存占用大：**Full 协方差需要存储 k 个 $d \times d$ 的矩阵，在高维数据上内存开销显著（本实验通过 PCA 降维到 50 维来缓解此问题）。
3. **对初始化敏感：**实验结果表明，不同的初始化方法会导致显著不同的结果，且可能陷入较差的局部最优。
4. **协方差矩阵的数值稳定性：**在高维或样本不足的情况下，协方差矩阵可能接近奇异，需要正则化处理（本实验使用 10^{-4} 的正则化系数）。

3、对数似然与聚类精度的关系讨论

实验中出现的一个重要现象是：对数似然与聚类精度（ACC）并不总是正相关。具体表现为：

- 在协方差结构对比中，两者呈现正相关：Full 协方差的对数似然和 ACC 都最高。
 - 在初始化方法对比中，两者呈现负相关：K-Means++ 初始化的对数似然更高，但 ACC 更低。
- 这一现象的根本原因在于：
1. **优化目标不同：**对数似然衡量的是模型对数据概率分布的拟合程度，而 ACC 衡量的是标签一致性。这两个目标并不等价。
 2. **标签的人为性：**MNIST 数据集的标签是基于人类对数字的认知定义的。从纯数据分布的角度看，某些数字（如 4 和 9、3 和 8）在像素空间中可能存在重叠，模型可能会学习到与人类认知不同的划分方式。
 3. **局部最优的多样性：**GMM 的目标函数是非凸的，存在多个局部最优解。不同的局部最优解可能在似然和 ACC 上表现出不同的偏向。

实践建议：

- 如果任务目标是最大化标签一致性（如半监督学习的预聚类），应该使用 ACC 作为主要评价指标，并尝试多种初始化方法和随机种子。
- 如果任务目标是学习数据的概率分布（如异常检测、密度估计），应该使用对数似然作为评价指标。
- 建议同时报告对数似然和 ACC，并进行多次实验（不同随机种子）以评估结果的稳定性。

4、训练技巧与改进建议

基于实验过程中的观察，总结以下训练技巧：

1. **降维预处理：**对于高维数据（如 MNIST 的 784 维），使用 PCA 降维到 50-100 维可以显著提高训练速度，同时保留 80% 以上的方差信息。
2. **正则化：**在协方差矩阵的对角线上添加小的正则化项（如 10^{-4} ）可以避免矩阵奇异，提高数值稳定性。

3. **收敛判据:** 使用对数似然的绝对增益作为收敛判据比相对增益更合理。根据实验, 设置 $\text{tol} = 0.5$, $\text{patience} = 5$ 可以在保证收敛的同时避免过早停止。
4. **多次运行:** 由于初始化的随机性, 建议使用不同的随机种子运行多次 (如 5-10 次), 报告平均性能和标准差。
5. **早停策略:** 对于关注 ACC 的任务, 可以在训练过程中监控 ACC, 在其开始下降时停止训练 (虽然对数似然仍在上升)。

四、结论

本实验在 MNIST 手写数字数据集上系统地比较了 K-Means 和 GMM 两种聚类算法, 得到以下主要结论:

1. **GMM 优于 K-Means:** 在聚类精度上, GMM (Full 协方差 + Random 初始化) 达到 63.50%, 显著高于 K-Means 的 59.37%。GMM 的软聚类和灵活的协方差结构使其能够更好地适应数据的真实分布。
2. **协方差结构的重要性:** 在 GMM 中, Full 协方差矩阵的性能 (ACC 57.63%) 明显优于 Spherical (53.76%) 和 Diagonal (40.95%)。这说明捕捉维度间的相关性对于聚类性能至关重要。
3. **初始化的影响复杂:** 初始化会影响模型的性能和收敛速度。从本次实验来看, 随机初始化训练出来的模型不一定就会比其他复杂的初始化方法差, 但收敛速度一般会较慢。
4. **对数似然与 ACC 的不一致性:** 对数似然 (概率模型拟合) 和聚类精度 (标签一致性) 是两个不同的评价指标, 优化其中一个并不一定优化另一个。在实际应用中应根据任务目标选择合适的评价指标。
5. **计算效率的权衡:** K-Means 虽然性能略低, 但其计算效率远高于 GMM, 在资源受限或实时性要求高的场景下仍有价值。GMM 的 Full 协方差在降维后 (50 维) 的训练时间仍是 K-Means 的 2-3 倍。
6. **降维的必要性:** PCA 降维不仅能显著提高训练速度, 还能使 Full 协方差矩阵的使用成为可能。在保留 82% 方差的情况下, 从 784 维降到 50 维使得 GMM 的训练时间从不可行变为可接受。